Introduction to Optimization

Checkpoint 3: Constrained Component
Storyline B — Cloud Resource Allocation (Variant 2: Zones)

Team T12 (Section 3)

Tilesbay Yermukhamed Rakhima Noken Tamerlan Aitzhanov
Zere Zhussupbekova Danial Kabdulla

Data: `cp34_instance(12, 'B', 2)`

Repository	`https://github.com/zere90/12-checkpoint2`
Tag	`checkpoint3`
Commit hash	*(run `git rev-parse HEAD` after the final push, paste it here)*

Name ↔ role

Role	Member	Code authored	Task / Table	Section / hand task
M1	Tilesbay Yermukhamed	`src/model/`	Task 1	S1, H1
M2	Rakhima Noken	`src/lp/`	Task 2, Table 1	S2, H2
M3	Tamerlan Aitzhanov	`src/kkt/`	Task 3, Table 2	S3, H3
M4	Danial Kabdulla	`src/sens/`	Task 4, Table 3	S4, H4
M5 (project lead)	Zere Zhussupbekova	`experiments/run_all.py`, tables, PDF	Task 5	S5, H5

Declarations

External code: `src/lp/solve.py` uses `scipy.optimize.linprog(method= "highs")` as the LP solver, as explicitly permitted by Section 4 ("Any LP solver"); every other module (`src/model/`, `src/kkt/`, `src/sens/`, `src/bound.py`) is written from scratch with `numpy` and the standard library only. **AI assistance:** we consulted an AI assistant (Claude) for help while working on this checkpoint. All code was read, run and verified by the author before committing.

1 Table 1 — model, LP value and fractionality

Computed instance values match the task file exactly: S=10 servers, M=35 VMs, $\sum$cpu_cap=454, $\sum$mem_cap=1680, $\sum$cpu_req=172, $\sum$mem_req=615, 60 forbidden (zone-mismatched) pairs kept with upper bound 0, giving 360 variables and 95 constraint rows (35 of (B1) + 10 of (B2) + 10 of (B3) + 40 of (B4); bounds not counted, per Task 1).

Quantity	Value
Variables n	360 (325 of y_{kl}, 10 of t_l; 60 fixed to 0)
Constraint rows m	95
z_{LP}	5.209695
Solver	`scipy.optimize.linprog(method="highs")`
Time	0.0026 s
Fractional y_{kl} $(0 < y_{kl} < 1)$	19, spanning 9 of the 35 VMs

The optimal vertex is not unique in general, so the exact fractional set can depend on the solver and even on the exact solver version; z_{LP} does not. The 9 split VMs are $k \in \{0, 13, 14, 18, 19, 23, 25, 29, 31\}$: eight are split across exactly *two* servers (e.g. VM 0: 0.5375 on server 0, 0.4625 on server 6), and one (VM 23) is split across *three* servers (0.3066, 0.0625, 0.6309 on servers 3, 4, 6). Full list in `results/table1_fractional.csv`.

2 Table 2 — optimality conditions (a)–(e)

Condition	Value	Satisfied ($\leq 10^{-6}$)
(a) primal feasibility, max violation	8.9×10^{-16}	yes
(b) dual feasibility (signs), max violation	0	yes
(c) complementary slackness, max $\lvert\pi_i \cdot \text{slack}_i\rvert$	1.1×10^{-16}	yes
(d) reduced costs, max violation	8.8×10^{-17}	yes
(e) strong duality, $\lvert\text{gap}\rvert$	8.9×10^{-16}	yes
All five satisfied		**yes**

3 Table 3 — sensitivity of the designated constraint

Designated row (Task 4): (B1) for k^*=13, the VM with the largest CPU request (cpu_req[13]=8): $\sum_l y_{13,l} = 1$, $b_0 = 1$.

Quantity	Value
$z(b_0 - 0.01)$	5.205338
$z(b_0)$	5.209695
$z(b_0 + 0.01)$	5.214052
Left slope	0.435730
Solver's dual price π	0.435730
Right slope	0.435730
Interval check (left $\leq \pi \leq$ right)	passed
Degenerate?	no

4 S1 — The model and what it ignores

Sets and parameters. $k = 0, \ldots, M-1$ indexes VMs, $l = 0, \ldots, S-1$ indexes servers; $\text{cpu_req}_k, \text{mem_req}_k$ are VM k's requirements; C_l, R_l are server l's capacity; $\text{zone}_l, \text{vm_required_zone}_k$ give the zone restriction (-1: none). **Variables.** $y_{kl} \in \{0, 1\}$: VM k runs on server l; $t_l \geq 0$: server l's latency. **Objective.** $\min \sum_l t_l$ — total latency, not cost or server count. **Constraints.** (B1) every VM placed on exactly one server; (B2)/(B3) a server's total CPU/memory demand cannot exceed its capacity; (B4) t_l is at least every one of 4 tangent lines of the convex $g(u) = u/(1-u)$ (an M/M/1-style latency model), so t_l is a piecewise-linear lower envelope of the true nonlinear latency in CPU utilisation u_l. **Variant 2 (zones).** if VM k needs a zone and server l is in a different zone, y_{kl} is forbidden; we keep that variable with ub = 0 rather than deleting it, so the variable count matches the check value (360, not 300). **What it ignores** (Section 2's own list, applied to our variant): the true nonlinear M/M/1 latency is replaced by a 4-piece linear lower bound; there is no routing, no warehouse trip, no arrival order; memory and CPU are the only two resources modelled; and the model is static — it places every VM once, with no migration cost.

Signed: Tilesbay Yermukhamed

5 S2 — Value and fractionality of the LP

Solving the LP relaxation (Table 1) with `scipy.optimize.linprog(method="highs")` takes 2.6 ms and gives $z_{LP} = 5.209695$. Of the 325 placement variables, 19 are strictly fractional, touching 9 of the 35 VMs (26%); the other 26 VMs are placed wholly on one server, exactly as in an integer solution. The fractional VMs are not a uniform smear: eight split 2-ways and one (VM 23) splits 3-ways, and in most cases one share dominates (e.g. VM 25: 0.8125/0.1875) rather than an even $1/n$ split — the LP is pushing as much of each VM onto its cheapest-latency server as the two capacity rows allow, only using a second (or third) server for the residual that would otherwise tip that server's utilisation past a cheaper tangent segment of (B4). This is the behaviour Task 5 asks us to read: fractional placement is concentrated where (B4)'s piecewise-linearity, not raw capacity, is the binding force (S5, active-row table). The optimal vertex returned by `highs` is one of possibly several optima with the same z_{LP} — a different solver, or even a different solver version, could fractionalise a different subset of VMs without changing $z_{LP} = 5.209695$.

Signed: Rakhima Noken

6 S3 — Is the solution optimal, and how do we know?

Before trusting any dual price we fixed our sign convention on a two-variable LP with a known answer: $\min x_1 + x_2$ s.t. $x_1 + x_2 \geq 1$, $0 \leq x \leq 1$, whose optimum is $z = 1$ at, e.g., $x = (1, 0)$, with a unique dual price of $+1$ on the single "$\geq$" row. `src/lp` returns exactly $\pi = +1$ for that row, confirming our convention: $\pi_i \geq 0$ for "$\geq$" rows, $\pi_i \leq 0$ for "$\leq$" rows, free for "$=$" rows (`tests/test_kkt.py`, `test_two_variable_sign_convention_sanity_check`) — the same convention Table 2 is built on. With that settled, Table 2 shows all five KKT conditions hold on our real B2 solution to at most 8.9×10^{-16}, many orders of magnitude inside the 10^{-6} tolerance: this is not "the solver said so," it is a self-contained numerical certificate — primal feasibility, correct dual signs, complementary slackness on every one of the 95 rows, zero reduced cost on every strictly-fractional variable, and $c^\top x = b^\top \pi + \sum_j \min(0, r_j)$ exactly, which is strong duality closing the primal-dual gap to machine precision. `src/kkt/checker.py` is a plain `(c, A, b, sense, lb, ub, x, pi)` function with no dependency on how x, π were produced, so it would equally certify another team's solver's output (as Task 3 requires).

Signed: Tamerlan Aitzhanov

7 S4 — What the dual price says about the storyline

The designated row is (B1) for VM 13 — the VM with the single largest CPU request (cpu_req=8) — forced to be placed exactly once. Its dual price is $\pi = 0.435730 > 0$, and Table 3's two finite-difference slopes agree with it to six decimals, so the row is not degenerate: this is a trustworthy, locally exact number, not an artifact of one vertex. Because the row is an equality, π is free in sign, and a positive value says relaxing it upward (as if VM 13 needed to be "placed" $1+\varepsilon$ times, e.g. requiring a hot spare) would raise total latency by about 0.436 per unit of ε — our single biggest VM is genuinely expensive to accommodate, because wherever it lands it pushes that server's utilisation measurably up the convex (B4) curve. In storyline terms: π_{13} is the marginal latency cost of over-provisioning placement for the heaviest VM in the fleet, and its sign (positive, not zero or negative) tells us this VM is never a "free rider" sitting in a mostly-empty server — every server it could be placed on is already latency-sensitive at the margin. A π of exactly 0 would instead mean some server has so much CPU headroom left that an extra unit of this requirement costs nothing; we do not see that here.

Signed: Danial Kabdulla

8 S5 — The bound and the bridge to Checkpoint 4

$z_{LP} \le z^*$ **in four lines.** Every integer-feasible $y \in \{0,1\}^n$ (with the box-bound $0 \le t$) satisfies $0 \le y \le 1$ as well, so it is also feasible for the LP relaxation; the LP's feasible set is therefore a superset of the integer-feasible set, over the *same* objective $\sum_l t_l$. Minimizing a fixed function over a larger set can only give a value less than or equal to minimizing it over a subset, so $z_{LP} = \min_{\text{LP-feas.}} \le \min_{\text{int-feas.}} = z^*$. **What the gap measures.** For a feasible integer z (e.g. a Checkpoint-4 heuristic's output), $(z - z_{LP})/z$ measures how far *that particular solution* is from the LP's lower bound — a practical, computable proxy for "how close to optimal is this," since the true z^* is unknown in Checkpoint 4. **What it does not tell you**: whether z is itself optimal (a large gap could mean a weak heuristic *or* a loose LP bound, and this number alone cannot distinguish the two causes) or how loose z_{LP} is relative to z^* specifically.

Active rows per family.

Family	Active	Total	% active
B1 (placement, =)	35	35	100%
B2 (CPU capacity)	0	10	0%
B3 (memory capacity)	1	10	10%
B4 (latency, piecewise-linear)	19	40	48%

B1 is trivially always tight (an equality). The real story is B2 vs. B4: not a single CPU-capacity row binds, and only one memory row does, but nearly half of the 40 latency tangent constraints are active — **the piecewise-linear latency family (B4), not raw CPU/memory capacity, drives the cost**, consistent with S4's reading of π_{13}.

Prediction for Checkpoint 4. With 19 of 325 placement variables (9 of 35 VMs) fractional at the LP optimum, and one of those split three ways rather than cleanly rounding, we predict the LP bound will *not* be perfectly tight: rounding these fractional VMs to integers will push some server past a cheaper (B4) tangent segment, costing some latency above z_{LP}, so we expect a modest but nonzero gap $(z^* - z_{LP})/z^*$ once Checkpoint 4 finds z^* (or a heuristic z) exactly.

Signed: Zere Zhussupbekova

9 Synthesis

What the LP tells us about this instance. Capacity is abundant (B2: 0/10 active, B3: only 1/10) but latency is not: B4's 48% activity rate and $\pi_{13} = 0.436 > 0$ on our single largest VM both point the same way — this instance is *latency-bound*, not *resource-bound*. A capacity-only view of this fleet ("do the servers have room?") would wrongly conclude the placement problem is nearly slack; it is the convex latency penalty, not headroom, doing almost all of the constraining. The 9 fractional VMs at the LP optimum are concentrated, not spread evenly, which (S5) predicts a real but modest integrality gap rather than either a perfectly tight or wildly loose bound.

Modelling assumptions where the handout left a gap. (1) We chose `scipy`'s `highs` LP solver over PuLP/CBC, exactly as Section 4's "any LP solver" permits; this sidesteps the `pulp<4` pinning requirement but means our exact fractional vertex (not z_{LP}) is solver- and even solver-version-dependent, as noted in S2. (2) For Task 4 we perturbed the right-hand side of (B1) at $k^* = 13$ by ± 0.01 while re-solving the full LP from scratch (not a basis-only re-optimization), which is exact but does not by itself certify how far the interval of validity around $b_0 = 1$ extends — we only checked the two adjacent points, consistent with what Task 4 asks for. (3) We kept forbidden (zone-mismatched) variables in the model with upper bound 0 rather than deleting them, per the handout's explicit instruction, which costs nothing numerically but means 18% of our 360 columns (60/360) are structurally dead weight by construction. **What we would not trust from Checkpoint 1 any more:** any latency number from Checkpoint 1's exact nonlinear M/M/1 model, since (B4) is only a 4-segment lower envelope of it — our z_{LP} is a bound on a *simplified* objective, and the true nonlinear latency at the same placement would be at least as large as what we report here.

Appendix: hand tasks H1–H5

A note before these pages. The five sections below are typed summaries of the reference derivations (full detail in `T12_HandSolutions_CP3.pdf`); each member's own signed, worked-by-hand page is photographed/scanned and kept in `hand/` in the repository (not pasted into this PDF), exactly as for Checkpoint 2. The typed summaries and the scans agree; the scan in `hand/` is the version of record for I3.

H1 — Tilesbay Yermukhamed — signed

Matrix form: $c = (5,5,6)^\top$, $A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$, $b = (1,1,1)^\top$, s.t. $Ay \geq b$, $0 \leq y \leq 1$: 3 variables, 3 rows. Rewriting as "$\leq$" rows multiplies every row by -1; every dual price flips sign (a "$\geq$"-row price $\pi_i \geq 0$ becomes a "$\leq$"-row price $-\pi_i \leq 0$ for the same constraint). All 8 binary points checked against the 3 covering rows; 4 are feasible (every point with at least two $y_j = 1$), cheapest is $(1,1,0)$ with $z^* = \mathbf{10}$.

Signed: **Tilesbay Yermukhamed** — see `hand/H1_signed.pdf` in the repository.

H2 — Rakhima Noken — signed

$y = (1/2, 1/2, 1/2)$: every row $1/2 + 1/2 = 1 \geq 1$ (tight), $0 \leq 1/2 \leq 1$ — feasible, cost $= 5(1/2) + 5(1/2) + 6(1/2) = \mathbf{8}$. Round down $(0,0,0)$: infeasible (every row fails). Round up $(1,1,1)$: feasible, cost 16. $z^* = 10$ (H1), and solving the LP confirms $(1/2, 1/2, 1/2)$ is exactly the LP optimum, $z_{LP} = 8$; gap $(10-8)/10 = \mathbf{0.2}$ (20%). No integral point can be LP-optimal: every integer point costs $\geq 10 > 8 = z_{LP}$, and the LP optimum is the unique fractional point $(1/2, 1/2, 1/2)$, strictly cheaper than any point in $\{0,1\}^3$.

Signed: **Rakhima Noken** — see `hand/H2_signed.pdf` in the repository.

H3 — Tamerlan Aitzhanov — signed

At $x = (0.5, 0.5, 0.5)$, $\pi = (2,3,3)$: all five conditions (a)–(e) hold to 0 exactly — a genuine, certified optimal primal-dual pair. With $\pi' = (3,2,2)$: (a)–(c) are unaffected (they do not depend on which π we test), but reduced cost $r'_3 = 6 - (3 \cdot 0 + 2 \cdot 1 + 2 \cdot 1) = 2 \neq 0$ — condition (d) **fails by** 2 — and strong duality gives $b^\top \pi' = 7 \neq 8 = c^\top x$ — condition (e) **fails by** 1. π' is still sign-feasible, so by weak duality $b^\top \pi' + \sum \min(0, r'_j) = 7$ remains a valid lower bound on $z_{LP} = 8$, just **1** short of the tight bound achieved by the true dual.

Signed: **Tamerlan Aitzhanov** — see `hand/H3_2Tamerlan.pdf` in the repository.

H4 — Danial Kabdulla — signed

Raising (T1)'s RHS to $1 + \varepsilon$ with all three rows kept tight gives $y_1 = y_2 = 0.5 + \varepsilon/2$, $y_3 = 0.5 - \varepsilon/2$, and cost $z(1+\varepsilon) = 8 + 2\varepsilon$: slope 2, exactly $\pi_1 = 2$ from the task file. Valid for $-1 \leq \varepsilon \leq 1$ (from $0 \leq y \leq 1$); at $\varepsilon = \pm 1$ a variable hits its bound and the slope could change beyond that point. If (T1) means "the first customer must be covered," $\pi_1 = 2$ says tightening or relaxing that coverage requirement by one unit changes the team's optimal cost by exactly 2 per unit, in the valid range.

Signed: **Danial Kabdulla** — see `hand/H4_Sens_Kabdulla_Danial.pdf` in the repository.

H5 — Zere Zhussupbekova — signed

$z_{LP} \le z^*$ in three lines: every integer-feasible $y \in \{0,1\}^3$ also satisfies $0 \le y \le 1$, so it is LP-feasible too; minimizing the same cost over the larger LP-feasible set can only give a value $\le$ the minimum over the smaller integer-feasible subset, so $z_{LP} \le z^*$. Three gaps, using $z_{LP} = 8$: true optimum $(1,1,0)$, $z^* = 10$, $(10-8)/10 = \mathbf{0.2}$; feasible point $(1,0,1)$, cost 11, $(11-8)/11 \approx \mathbf{0.273}$; rounded-up point $(1,1,1)$, cost 16, $(16-8)/16 = \mathbf{0.5}$. The first (using z^*) measures the intrinsic quality of the LP bound itself; the other two (using a merely-feasible z) measure the quality of *that specific solution* — exactly the distinction Checkpoint 4 will need, where z^* is unknown and only z_{LP} and a heuristic's z are available.

Signed: **Zere Zhussupbekova** — see `hand/H5_Zhussupbekova_Zere_2.pdf` in the repository.

Contribution statement

To be reviewed and signed by all five members before submission; if it disagrees with the Git history, the Git history decides.

Member	Role	What they did	Share
Tilesbay Yermukhamed	M1	`src/model/`: data loading, matrix-form build, row/column bookkeeping; H1; S1	19%
Rakhima Noken	M2	`src/lp/`: LP solving, reading the solution, Table 1; H2; S2	19%
Tamerlan Aitzhanov	M3	`src/kkt/`: conditions (a)–(e) checker, sign-convention sanity test, Table 2; H3; S3	19%
Danial Kabdulla	M4	`src/sens/`: perturbation runs, interval check, Table 3; H4; S4	19%
Zere Zhussupbekova	M5	`experiments/run_all.py`, `src/bound.py`, assembling all tables, hand-solutions reference and this PDF; H5; S5; Synthesis	24%
Total			**100**%

Signatures:

Tilesbay Yermukhamed: _____

Rakhima Noken: _____

Tamerlan Aitzhanov: _____

Danial Kabdulla: _____

Zere Zhussupbekova: _____